CSCI 6461 | Computer Systems Architecture | TOPIC 6

Subroutines, Calling Conventions, & the Stack

Function linkage, register preservation rules, stack frame management, and AAPCS64 architectural compliance

Dr. J. Burns

Topic 6 Outline

1. **Function Calls and Returns**
   - Branch-with-link, the link register, returns, leaf functions, and non-leaf functions.
2. **AAPCS64 Register Rules**
   - Argument registers, return registers, caller-saved registers, and callee-saved registers.
3. **Stack Frames**
   - Stack growth, frame records, frame pointer use, prologues, epilogues, and local storage.
4. **Worked Function Examples**
   - Leaf functions, nested calls, recursion, extra arguments, and common linkage mistakes.

PART 1

Subroutine Linkage & Control Flow

High-Level Function Execution Model

- **Subroutines** provide modular abstraction, reusability, and clean software architecture.
- To execute a function call, the CPU must satisfy five core requirements:
  1. **Pass Arguments:** Transfer input parameters to the function.
  2. **Transfer Control:** Jump to the function's entry point address.
  3. **Isolate Local State:** Provide memory space for function local variables.
  4. **Return Value:** Pass computed results back to the caller.
  5. **Resume Execution:** Return seamlessly to the instruction following the call.

The Link Register (`x30`) and Branch-with-Link (`bl`)

- Invoking a function requires remembering where to return when it completes.
- **Branch with Link (`bl label`):** Atomically performs two operations:
  1. Saves return address (PC $+ 4$) into the **Link Register (`x30` / `lr`)**.
  2. Branches to target function: PC $\leftarrow$ label.

PC-Relative Calling Range

The `bl` instruction encodes a signed 26-bit immediate offset, allowing direct function calls within a ± 128 MB window of the current instruction.

Subroutine Return Mechanics (`ret`)

- **Return from Subroutine (`ret`):** Branches to the address held in a register. With no explicit operand, `ret` uses the link register `x30`:

$$\text{PC} \leftarrow x_{30}$$

- A normal `bl` / `ret` pair therefore resumes at the instruction following the original call.

Example: Minimal Leaf Return Sequence

```
caller_func:
  bl helper_func   // x30 = return address; branch to helper
  mov x1, x0       // resumes here after ret
helper_func:
  add x0, x0, #1   // compute result
  ret              // default target register is x30
```

Leaf vs. Non-Leaf Functions

Leaf Functions

- Call **no other functions**.
- `x30` is not overwritten by a nested `bl`.
- May still use stack space for locals or spills, but need not save `x30` solely for linkage.

Non-Leaf Functions

- Call one or more **nested subroutines**.
- A nested `bl` overwrites `x30` with the nested return address.
- Preserve the incoming return address before that happens if it will be needed later.

The Non-Leaf Rule

If a function makes another call and must later return to its own caller, its incoming return address must be preserved before `x30` is overwritten.

Indirect Subroutine Calls (`blr`)

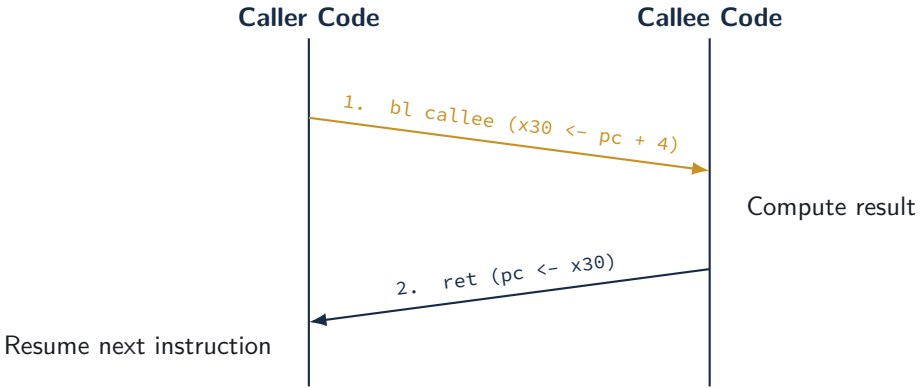
- When the target function address is determined dynamically at runtime, use **Branch with Link to Register (`blr xn`)**.
- Saves return address ($\text{PC} + 4$) into `x30` and jumps to address held in register `xn`.

Primary Applications

- **C Function Pointers:** `void (*func_ptr)(int) = my_callback;`
- **Virtual Method Tables (vtables):** C++ polymorphism and object dispatch.
- **Shared Library Veneers:** Dynamic linking and Symbol Jump Tables.

```
ldr x2, [x0, #8]   // Load function pointer from vtable
blr x2   // Call function indirectly via register x2
```

Subroutine Call Execution Trace



Control transfers from caller to callee, then returns using `x30`.

PART 2

Procedure Call Standard & Register Roles

Purpose of Procedure Call Standards (AAPCS64)

- **AAPCS64:** Procedure Call Standard for the ARM 64-bit Architecture.
- Defines the calling-convention contract between independently compiled caller and callee functions.
- **Why the ABI Contract Matters**
  - Allows assembly code to call C library functions such as `printf` and `malloc`.
  - Enables code generated by different compilers to interoperate.
  - Defines parameter passing, return values, stack use, and register-preservation rules.

Parameter Passing Registers (x0–x7)

- Up to **eight 64-bit scalar integer or pointer arguments** can be passed in x0 through x7, subject to AAPCS64 argument-classification rules.

Parameter Index	Register	C Language Example
1st Argument	x0 (w0)	int64_t a
2nd Argument	x1 (w1)	char *str
3rd Argument	x2 (w2)	size_t length
4th–8th Arguments	x3–x7	Additional scalar arguments

Additional Arguments

Arguments not assigned to registers are placed in the stack argument area according to AAPCS64 layout and classification rules.

Return Value Registers

- Many scalar return values are returned directly in registers.

Return Data	Register(s)	Typical Rule
32-bit scalar integer / Boolean	`w0`	Result returned in the lower 32-bit view.
64-bit scalar integer / pointer	`x0`	Result returned in register 0.
Suitable integer-like aggregate up to 16 bytes	`x0`, `x1`	May use two general-purpose result registers.
Larger or non-register aggregate	indirect via `x8`	Caller supplies an indirect result location when required.

Scope Note

Floating-point and SIMD return rules use the SIMD/FP register file and are outside this lecture's focus.

Caller-Saved Registers (`x0`–`x17`)

- Also called **scratch** or **volatile** registers.
- The ABI does not require the callee to preserve these registers across a call.

Registers	AAPCS64 Role	Preservation
`x0`–`x7`	Arguments and result registers	Caller-saved
`x8`	Indirect result-location register when required	Caller-saved
`x9`–`x15`	Temporary registers	Caller-saved
`x16`–`x17`	Intra-procedure-call temporaries (IP0/IP1)	Caller-saved

Caller Responsibility

A caller that needs a value in these registers after a call must preserve that value before executing `bl` or `blr`.

Callee-Saved Registers (x19–x28)

- Also known as **Preserved** or **Non-Volatile** registers.
- Registers x19 through x28 hold long-lived variables across subroutine invocations.

The Callee Contract

If a subroutine modifies any register in x19–x28, its incoming value must be preserved and restored before return.

Typical Preservation (Prologue)

```
str x19, [sp, #-16]!
```

Preserves caller's x19 before modifying it.

Typical Restoration (Epilogue)

```
ldr x19, [sp], #16
```

Restores original x19 before ret.

Special-Purpose ABI Registers

Register	ABI Name	AAPCS64 / ABI Role
`x8`	XR	Indirect result-location register when a return value is passed through memory.
`x16`, `x17`	IP0, IP1	Intra-procedure-call temporaries; linkers and veneers may use them.
`x18`	PR	Platform register; availability depends on the platform ABI.
`x29`	FP	Frame pointer when a frame record is maintained.
`x30`	LR	Link register; `bl`/`blr` write the return address here.
`sp`	SP	Stack pointer; must satisfy AAPCS64 alignment and stack-use rules.

Takeaway: Treat `x18`, `x29`, `x30`, and `sp` according to their ABI and linkage roles rather than as ordinary scratch registers.

Register Responsibility Summary

Registers	Role / Usage	Preservation Contract
`x0`–`x7`	Arguments and result values	Caller-saved
`x8`–`x17`	ABI-specific and temporary uses	Caller-saved
`x19`–`x28`	Preserved local values	Callee must restore if modified
`x29` (`fp`)	Frame pointer when used	Callee-saved
`x30` (`lr`)	Return address after `bl`/`blr`	A non-leaf callee preserves its incoming return address if needed after another call
`sp`	Stack pointer	Must be restored to the required value before return and obey alignment rules

PART 3

The Runtime Stack & Activation Frames

The Stack Memory Model

- The runtime stack is a dynamic memory region allocated in process address space.
- **Key Characteristics:**
  - Grows **downward** from high memory addresses toward low memory addresses.
  - Stack Pointer (sp) points to the current **lowest active address** (top of stack).
  - Operates on a Last-In, First-Out (LIFO) order.

Allocation vs. Deallocation

- **Push / Allocate:** Subtract from sp (`sub sp, sp, #32`).
- **Pop / Deallocate:** Add to sp (`add sp, sp, #32`).

What Is a Stack Frame?

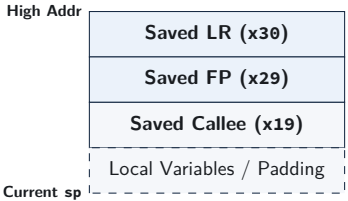
- Each function call may reserve a region of the runtime stack called an **activation frame** or **stack frame**.
- A frame can hold:
  - saved return and register state,
  - callee-saved registers,
  - local variables, temporary storage, and alignment padding.
- The stack pointer (sp) identifies the current stack boundary and changes as stack storage is allocated or released.

Why a Frame Pointer?

When a stable reference within the frame is useful, x29 can serve as the frame pointer while sp continues to track the current stack position.

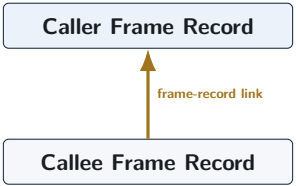
Example Activation Frame Structure

- An **activation frame** stores state associated with one function invocation.
- Common elements include:
  1. saved link register (`x30`) and frame pointer (`x29`),
  2. saved callee-saved registers,
  3. local variables and buffers.



Frame Pointer (x29) and Frame Records

- The **Frame Pointer (`x29` / `fp`)** can provide a fixed reference point within the activation frame for accessing arguments and local variables.
- **Frame Pointer Conventions:**
  - Under standard AAPCS64 profiles, functions may maintain a frame record linking saved `x29` and `x30`.
  - When used, this creates a traceable chain of stack frames to assist debuggers (e.g., GDB) with backtraces.
  - However, platform rules or optimization levels may permit omitting frame records or repurposing `x29`.



Function Prologue Mechanics

- A **prologue** commonly allocates stack space and saves state needed by the function.

Example Prologue Template

```
my_func:
    // Allocate 32 bytes and save FP/LR
    stp     x29, x30, [sp, #-32]!

    // x29 points to the frame record
    mov     x29, sp

    // Save callee-saved register used by the body
    str     x19, [sp, #16]
```

Pre-indexed `stp` combines stack-pointer writeback with the paired store of `x29`/`x30`.

Function Epilogue Mechanics

- The **Epilogue** tears down the frame and restores state before returning.
- Reverses the exact sequence performed by the prologue:

Standard Epilogue Template

```
// 1. Restore saved callee registers
ldr    x19, [sp, #16]

// 2. Restore original FP and LR; release stack frame
ldp    x29, x30, [sp], #32

// 3. Return to caller
ret
```

Post-indexed `ldp` performs a paired architectural load and writeback in one instruction to restore `x29`/`x30` and deallocate 32 stack bytes.

Local Variable Allocation on the Stack

C Function with Local Array

```
void process(void) {
    uint64_t buffer[2];
    buffer[0] = 10;
    buffer[1] = 20;
}
```

Illustrative Leaf Stack Allocation

```
process:
    sub  sp, sp, #16

    mov  x0, #10
    str  x0, [sp, #0]
    mov  x0, #20
    str  x0, [sp, #8]

    add  sp, sp, #16
    ret
```

Because this example is a leaf function, it does not need to save `x30` solely for return linkage. The 16-byte local allocation preserves stack alignment.

Stack Alignment and Padding Rules

- AAPCS64 requires sp to satisfy 16-byte alignment at public interfaces and when used for stack memory accesses.
- Frame allocation therefore rounds the required stack space up to a multiple of 16 bytes.

Calculating Frame Size

$$\text{Frame Size} = \text{align}_{16}(\text{saved state} + \text{local storage})$$

Required Space	Frame Size	Example Allocation
16B saved FP/LR	16 bytes	`stp x29, x30, [sp, #-16]!`
16B saved FP/LR + 20B locals	48 bytes	allocate a 48-byte frame and place saved state/locals within it
16B saved FP/LR + 32B locals	48 bytes	allocate a 48-byte frame

PART 4

Worked Assembly Examples & Patterns

Worked Example 1: Minimal Leaf Function

C Code: `int64_t add_three(int64_t a, int64_t b, int64_t c)`

AArch64 Assembly Implementation

```
// Inputs: x0 = a, x1 = b, x2 = c
add_three:
            add x0, x0, x1   // x0 = a + b
            add x0, x0, x2   // x0 = (a + b) + c
            ret              // result already in x0
```

- Calls no subroutines and needs no stack allocation.
- The function consists of three architectural instructions.

Worked Example 2: Non-Leaf Nested Function

C Code: `int64_t compute(int64_t x)`

```
return add_three(x, 10, 20);
```

AArch64 Assembly Implementation

```
// Input: x0 = x
compute:
          stp x29, x30, [sp, #-16]!   // save FP and LR
          mov x29, sp                 // establish frame record
          mov x1, #10                 // 2nd argument
          mov x2, #20                 // 3rd argument
          bl add_three                // nested call overwrites x30
          ldp x29, x30, [sp], #16     // restore FP/LR
          ret                         // return to caller
```

Worked Example 3: Recursive Factorial

C Logic

```
uint64_t factorial(uint64_t n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
}
```

Each invocation preserves its own return address and the current value of n across the recursive call.

AArch64 Assembly

```
factorial:
    cmp  x0, #1
    b.ls .Lbase

    stp  x29, x30, [sp, #-32]!
    mov  x29, sp
    str  x19, [sp, #16]
    mov  x19, x0
    sub  x0, x0, #1
    bl   factorial
    mul  x0, x0, x19
    ldr  x19, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret

.Lbase:
    mov  x0, #1
    ret
```

Worked Example 4: Passing More Than 8 Arguments

Example Assumption

This example uses ten 64-bit integer arguments. Arguments 1–8 use `x0`–`x7`; the remaining arguments are placed in the stack argument area.

Caller Side

```
// Prepare 9th and 10th arguments
mov  x9,  #100
mov  x10, #200
stp  x9, x10, [sp, #-16]!
bl   func_with_10_args
add  sp, sp, #16
```

Callee Side

```
func_with_10_args:
    // Args 1-8 are in x0-x7
    ldr  x9,  [sp, #0]   // 9th arg
    ldr  x10, [sp, #8]   // 10th arg
    ret
```

Common Function Linkage Bugs in Assembly

- **1. Overwriting the Return Address:** A nested `bl` overwrites `x30`; failing to preserve the incoming return address can return control to the wrong location.
- **2. Corrupting Callee-Saved Registers:** Modifying `x19`–`x28` without restoring them violates the calling convention.
- **3. Stack Misalignment:** Violating AAPCS64 stack-alignment rules breaks ABI assumptions and can fault where SP-alignment checking is enabled.
- **4. Unbalanced Stack Pointer:** Failing to restore `sp` correctly can corrupt stack-resident state and later restores.

SUMMARY & ASSESSMENT

Review and Self-Test Questions

Topic 6: Summary

- **Linkage Primitives:** `bl`/`blr` write the return address to `x30`; `ret` normally returns through `x30`.
- **AAPCS64 Register Contract:**
  - Scalar arguments commonly begin in `x0`–`x7`; most scalar results return in `x0`.
  - `x0`–`x17` are caller-saved; `x19`–`x28` are callee-saved.
- **Activation Frames:** Stack frames hold saved linkage state, preserved registers, and local storage while maintaining required alignment.
- **Prologue/Epilogue:** Paired `stp`/`ldp` instructions commonly save and restore `x29`/`x30` while updating `sp`.

Self-Test Questions: Linkage & AAPCS64

1. **Leaf vs. Non-Leaf Function Linkage:** How do leaf and non-leaf functions differ regarding link-register management and return-address preservation?
2. **Register Preservation Contracts:** Suppose function `foo()` calls `bar()`. If `foo()` needs to maintain a loop counter value across the call, should it store that counter in `x9` or `x19`? Why?
3. **Parameter Passing Limits:** How are integer parameters handled when calling a C function declared with 10 scalar arguments: `void test(int a1, ..., int a10)`?

Self-Test Questions: Stack Frame Management

1. **Stack Allocation Alignment:** A non-leaf function saves `x29`/`x30` (16 bytes) and needs 20 bytes of local buffer space. What is the minimum frame size it must allocate on the stack to maintain AAPCS64 compliance?
2. **Prologue/Epilogue Verification:** Identify the bug in the following epilogue:

```
ldp x29, x30, [sp], #16
ldr x19, [sp, #16]
ret
```

3. **Frame Pointer Chains:** When frame records are maintained, how does saved `x29` help a debugger reconstruct the call chain?

Looking Ahead

Next Lecture Preview

Our Next Topic: Machine Code & AArch64 Instruction Encoding

- **32-Bit Instruction Words:** Relating assembly syntax to fixed-width A64 machine instructions.
- **Encoding Fields:** Identifying opcode, register, immediate, and shift fields within instruction formats.
- **Assembly to Machine Code:** Building and interpreting representative AArch64 instruction encodings.